Javascript

Objectives

Coverage

- Objects and Arrays
  - Properties and Methods
  - Browser Object Hierarchy
  - Addressing Objects
  - Creating Objects
  - Objects and Variables
  - External Objects
  - Security Implications
  - Arrays and Objects

Objectives (Contd.)

Coverage (Continued)

- Document Object Model
  - Introduction to DOM
  - The Navigator Object
  - The Window Object
  - Communicating between windows
  - Working With Frames in the DOM
  - The Document object
  - The Location & History Object
  - Image Object
  - Link Object

Objectives (Contd.)

Coverage (Continued)

- Form Validation The Form Object Model
  - Recap of Form Elements
  - Form Tag Events
  - Controlling Submission
  - Forms as Navigation
  - List Events
  - Buttons
  - Text Processing

Objectives (Contd.)

Day–3 Coverage (Continued)

- Cookies
  - Introduction to HTTP Cookies
  - Formatting Cookies
  - Persisting Cookies
  - Cookie Security
  - JavaScript & Cookies
  - Reading / Writing Cookies
  - Modifying and deleting Cookies

Introduction to Java Scripts

- **What is JavaScript???**
  - **JavaScript is a scripting Language created by Netscape**



- **Scripting Language is a lightweight programming language.**
- **Scripting Languages are not needed to be compiled.**
- **The language is interpreted at runtime.**

Introduction to JavaScript (Contd.)

- A JavaScript is usually directly embedded in an HTML page.
- External JavaScripts can be created which can be used by HTML pages.
- JavaScript adds interactivity to HTML pages.
- JavaScript's are integrated into the browsing environment.

Objects
&
Arrays

Dipendu Mishra

JavaScript Objects

- JavaScript is not a true Object Oriented language as C++ or Java but rather an Object Based language.
- Objects in JavaScript are software entities such as the browser window or an HTML document.

JavaScript Objects (contd.)

JavaScript supports the following constructs that you can use to create an object:

- ✔ the new operator
- ✔ the object initializer
- ✔ the constructor

JavaScript Objects (contd.)

The syntax of new operator:

```
var objectName = new Object()
```

The syntax of object initializer

```
objectName = {property1:value1,
property2:value2 , … ,
propertyN:valueN}
```

The syntax of constructor function

```
var objectName = new
FunctionName()
```

JavaScript Objects (contd.)

If you use the constructor function to create your own JavaScript object, you must fist define the object type by creating a function.

Then you create an instance of the new object type with the new operator.

Example

```
function
Computer(CPU,RAM,HD,OS)
{
  this.CPU = CPU
  this.RAM = RAM
  this.HD = HD
  this.OS = OS
 }
```

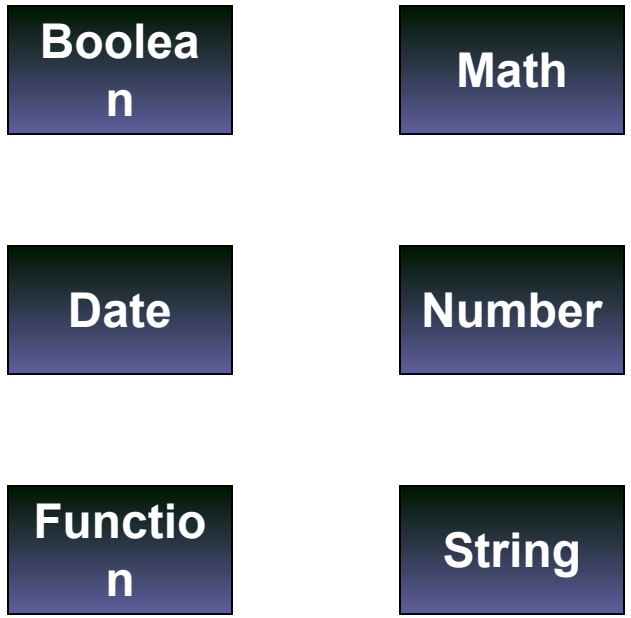
To create an object of Computer function

```
var c1 = new Computer
("PentiumIV", 256,40,
"WindowsXP")
```

JavaScript Core Objects (contd.)

JavaScript supports a range of predefined core objects:

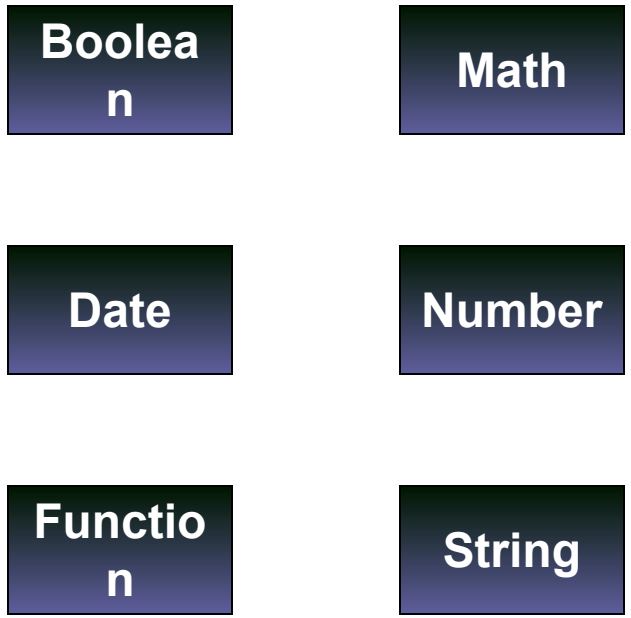
1. Boolean
2. Date
3. Function
4. Math
5. Number
6. String



JavaScript Core Objects (contd.)

JavaScript supports a range of predefined core objects:

1. Boolean
2. Date
3. Function
4. Math
5. Number
6. String



JavaScript Core Objects (contd.)

- **Boolean Object**
  - **This object is used to convert a non boolean value to a boolean value.**
  - **The Boolean Object is an Object Wrapper for a Boolean value**
  - **Boolean object is defined with the new keyword**

```
var BooleanObj=new Boolean()
```

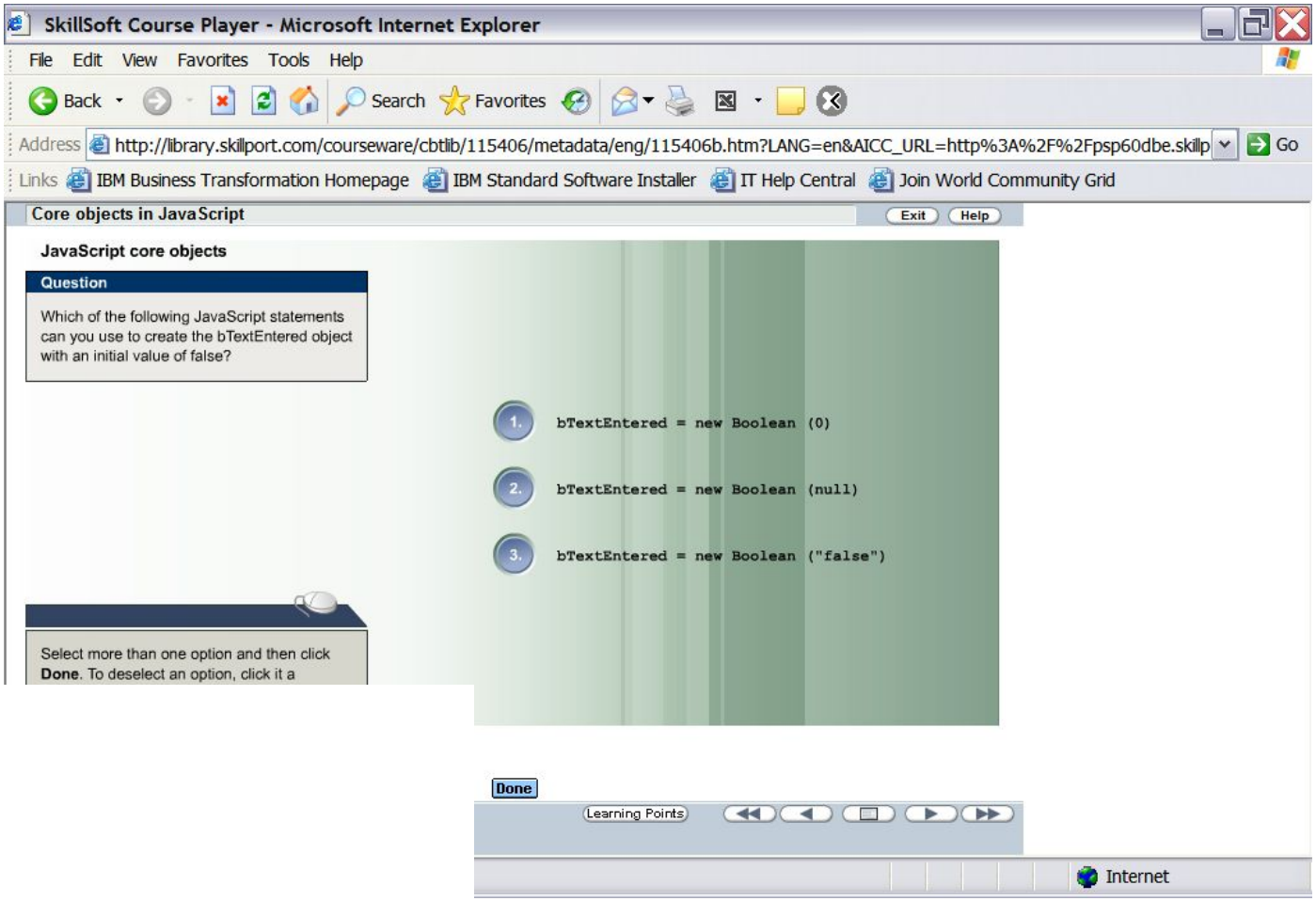
JavaScript Core Objects (contd.)

- All the following lines of code create Boolean objects with an initial value of false :
  - var myBoolean=new Boolean()
  - var myBoolean=new Boolean(0)
  - var myBoolean=new Boolean(null)
  - var myBoolean=new Boolean("")
  - var myBoolean=new Boolean(false)
  - var myBoolean=new Boolean(NaN)

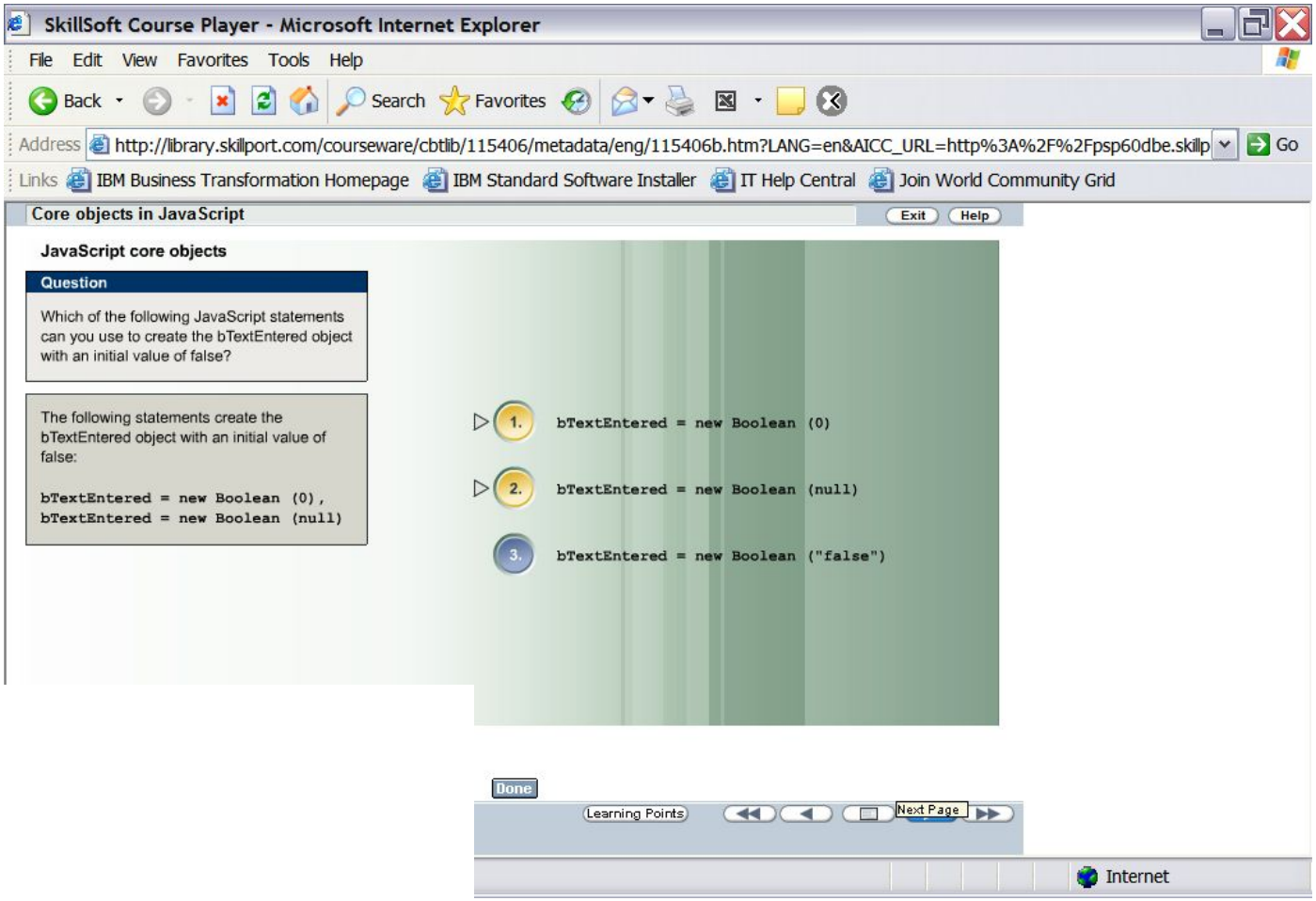
JavaScript Core Objects (contd.)

- All the following lines of code create Boolean objects with an initial value of true:
  - var myBoolean=new Boolean(true)
  - var myBoolean=new Boolean("true")
  - var myBoolean=new Boolean("false")
  - var myBoolean=new Boolean("Richard")

Test Your Understanding



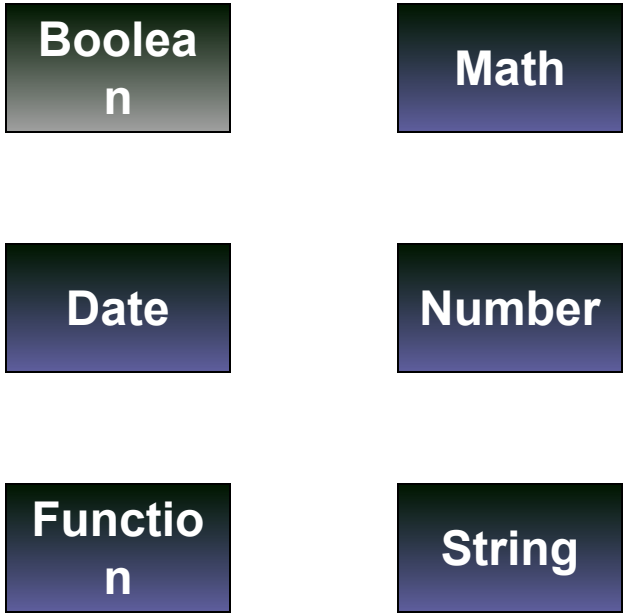
Test Your Understanding



JavaScript Core Objects (contd.)

JavaScript supports a range of predefined core objects:

1. Boolean
2. Date
3. Function
4. Math
5. Number
6. String



JavaScript Core Objects (contd.)

- Date Object
  - The Date object is used to work with dates and times.
  - An instance of the Date object with the "new" keyword.
  - An instance of Date object can be created as:

    var myDate=new Date()

    var myDate=new Date("Month dd, yyyy hh:mm:ss")

    var myDate=new Date("Month dd, yyyy")

    var myDate=new Date(yy,mm,dd,hh,mm,ss)

    var myDate=new Date(yy,mm,dd)

    var myDate= new Date(milliseconds)

JavaScript Core Objects (contd.)

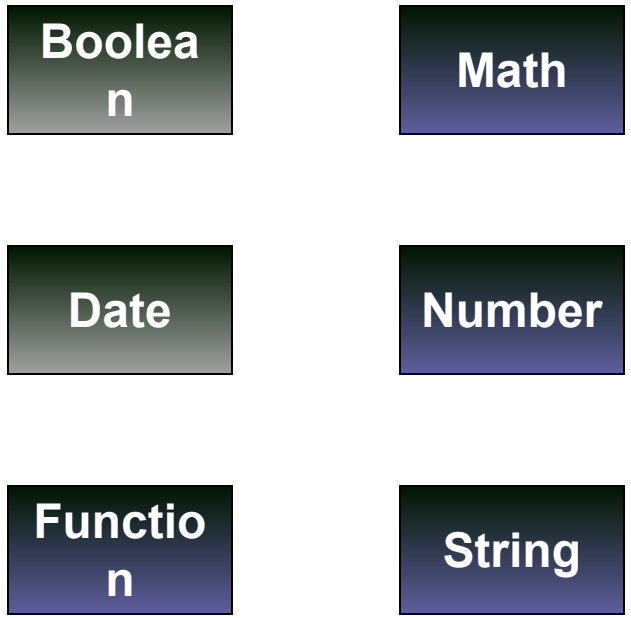
Commonly used methods of the Date Object

Methods	Description
Date()	Returns a Date object
getDate()	Returns the date of a Date object (from 1-31)
getDay()	Returns the day of a Date object (from 0-6. 0=Sunday, 1=Monday, etc.)
getMonth()	Returns the month of a Date object (from 0-11. 0=January, 1=February, etc.)
getFullYear()	Returns the year of the Date object (four digits)
getHours()	Returns the hour of the Date object (from 0-23)
getMinutes()	Returns the minute of the Date object (from 0-59)
getSeconds()	Returns the second of the Date object (from 0-59)

JavaScript Core Objects (contd.)

JavaScript supports a range of predefined core objects:

1. Boolean
2. Date
3. Function
4. Math
5. Number
6. String



JavaScript Core Objects (contd.)

Example

```
function
Computer(CPU,RAM,HD,OS)
{
  this.CPU = CPU
  this.RAM = RAM
  this.HD = HD
  this.OS = OS
 }
```

To create an object of Computer function

```
var c1 = new Computer
("PentiumIV", 256,40,
"WindowsXP")
```

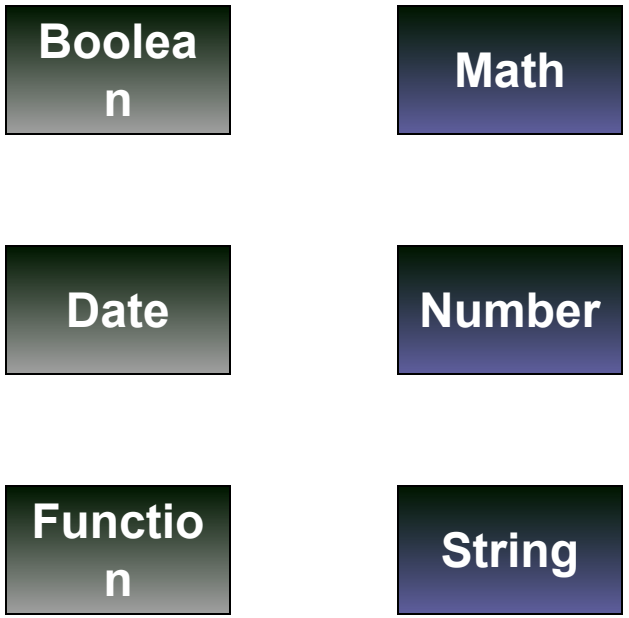


In the previous slides we have discussed this, Here c1 is the function obj

JavaScript Core Objects (contd.)

JavaScript supports a range of predefined core objects:

1. Boolean
2. Date
3. Function
4. Math
5. Number
6. String



JavaScript Core Objects (contd.)

- **Math Object**
  - **Math object allows to perform common mathematical functions.**
  - **The Math object includes several Mathematical values and functions.**
  - **The Math object need not be defined before using it.**

JavaScript Core Objects (contd.)

- **Mathematical values provided by JavaScript**
  - **Math.E**
  - **Math.PI**
  - **Math.SQRT2**
  - **Math.SQRT1_2**
  - **Math.LN2**
  - **Math.LN10**
  - **Math.LOG2E**
  - **Math.LOG10E**

JavaScript Core Objects (contd.)

Methods of Math object

method name	Description
abs(x)	Returns absolute value of x.
acos(x)	Returns arc cosine of x in radians.
asin(x)	Returns arc sine of x in radians.
atan(x)	Returns arc tan of x in radians.
atan2(y, x)	Counterclockwise angle between x axis and point (x,y).
ceil(x)	Returns the smallest integer greater than or equal to x. (round up).
cos(x)	Returns cosine of x, where x is in radians.
exp(x)	Returns e^x
floor(x)	Returns the largest integer less than or equal to x. (round down)

method name	Description
log(x)	Returns the natural logarithm of x.
max(a, b)	Returns the larger of a and b.
min(a, b)	Returns the lesser of a and b.
pow(x, y)	Returns X^y.
random()	Returns a pseudorandom number between 0 and 1. Example(s).
round(x)	Rounds x up or down to the nearest integer. It rounds .5 up. Example(s).
sin(x)	Returns the Sin of x.
sqrt(x)	Returns the square root of x.
tan(x)	Returns the Tan of x.

Test Your Understanding

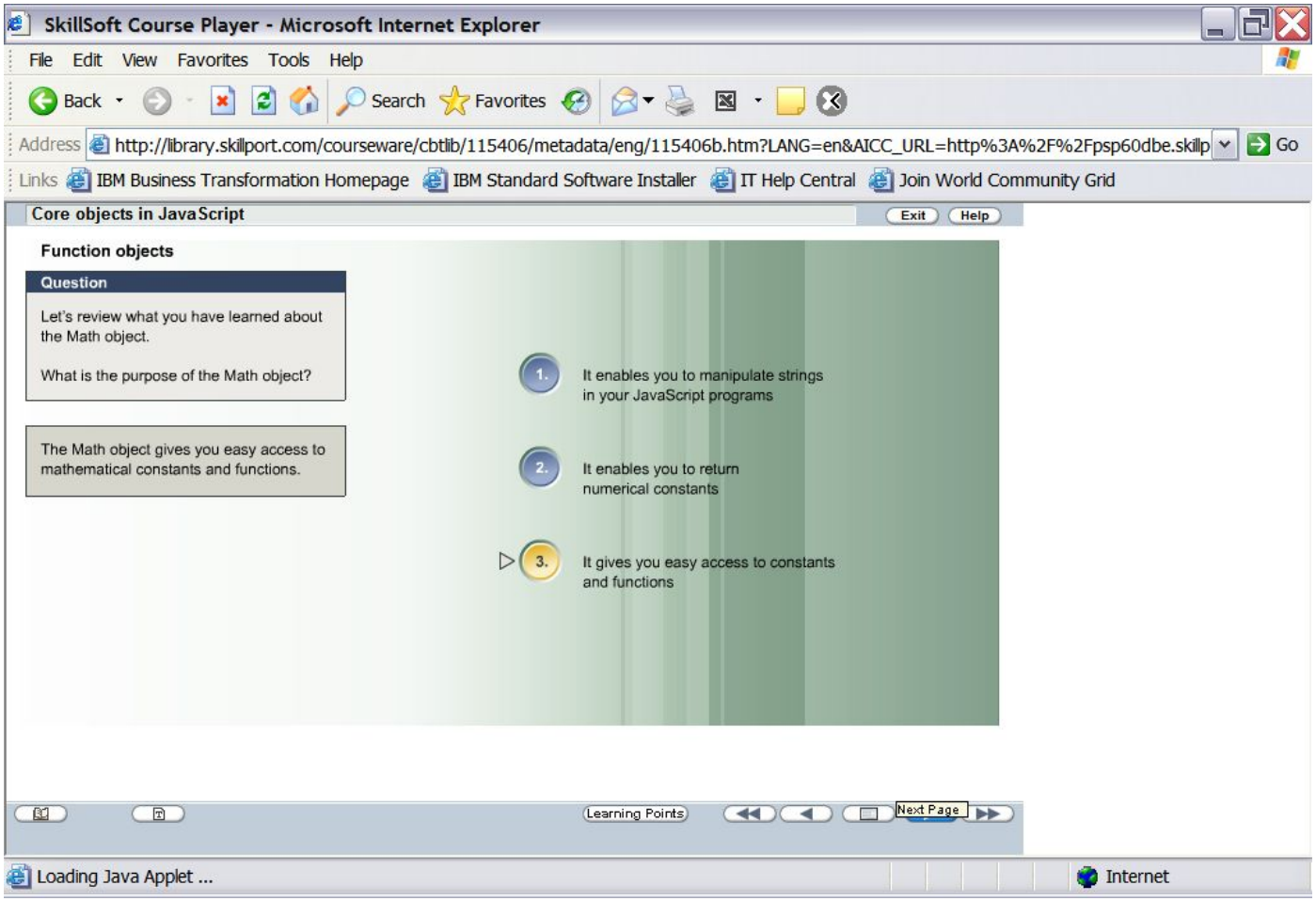
Question

Let's review what you have learned about the Math object.

What is the purpose of the Math object?

1. It enables you to manipulate strings in your JavaScript programs
2. It enables you to return numerical constants
3. It gives you easy access to constants and functions

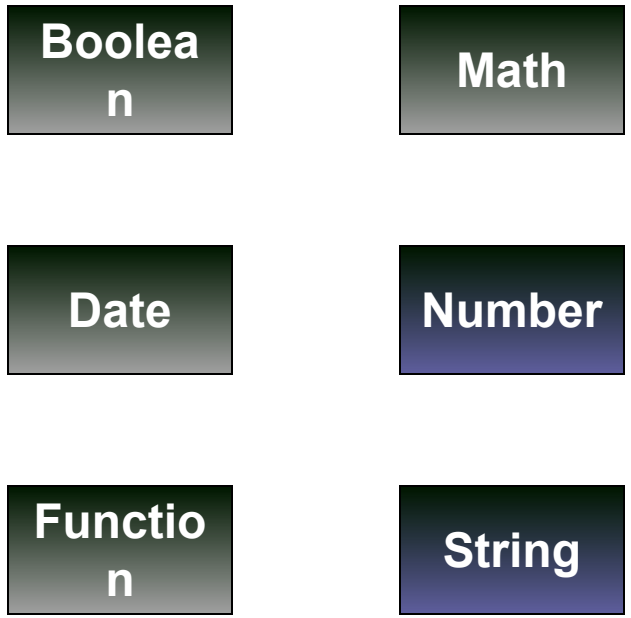
Test Your Understanding



JavaScript Core Objects (contd.)

JavaScript supports a range of predefined core objects:

1. Boolean
2. Date
3. Function
4. Math
5. Number
6. String



JavaScript Core Objects (contd.)

Points to be noted in regard to Number object:

- ✔ The JavaScript Number object returns numerical constants.
- ✔ You can't change the value of the number object's properties.
- ✔ Number object properties are called static properties.

JavaScript Core Objects (contd.)

Properties of Number object:

- ✔ MIN_VALUE
- ✔ MAX_VALUE
- ✔ NaN
- ✔ NEGATIVE_INFINITY
- ✔ POSITIVE_INFINITY

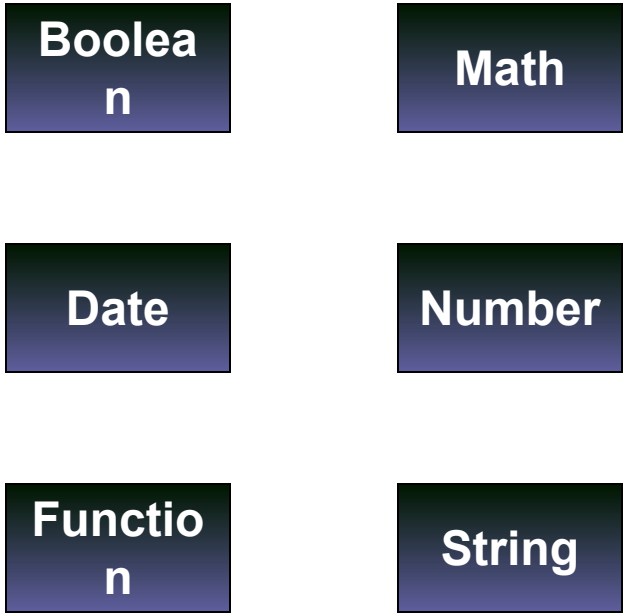
Example:

```
// Overflow trap
Car toohigh;
If(toohigh>=Number.MAX_VALUE)
{
  take necessary steps to reduce the vale
}
```

JavaScript Core Objects (contd.)

JavaScript supports a range of predefined core objects:

1. Boolean
2. Date
3. Function
4. Math
5. Number
6. String



JavaScript Core Objects (contd.)

- String object
  - The String object is used to manipulate a stored piece of text.
  - String objects must be created using the new keyword.
  - JavaScript automatically converts the string primitive to a temporary String object
  - A string literal can be embedded within a single or double quotation marks.

JavaScript Core Objects (contd.)

- String primitives and String objects give different results when evaluated as JavaScript.
- Primitives are treated as source code, but String objects are treated as a character sequence object.
  - s1 = “2 + 2” // creates a string primitive
  - s2 = new String("2 + 2") // creates a String object
  - eval(s1) // returns the number 4
  - eval(s2) // returns the string “2 + 2”
  - eval(s2.valueOf()); // returns the number 4

JavaScript Core Objects (contd.)

Common Methods of String Object

Method	Description
charAt()	Returns the character at a specified position
concat()	Joins two or more strings
indexOf()	Returns the position of the first occurrence of a specified string value in a string
lastIndexOf()	Returns the position of the last occurrence of a specified string value, searching backwards from the specified position in a string
match()	Searches for a specified value in a string
replace()	Replaces some characters with some other characters in a string
search()	Searches a string for a specified value
split()	Splits a string into an array of strings
substr()	Extracts a specified number of characters in a string, from a start index
substring()	Extracts the characters in a string between two specified indices
toLowerCase()	Displays a string in lowercase letters

heet1 Sheet2 Sheet3

Defining Arrays

- An Array object is used to store a set of values in a single variable name.
- An Array object is created with the new keyword.
- An array can be created as:
  - var MyArray=new Array()
- An array can also be created by specifying the array size.
  - var MyArray=new Array(3)

Arrays (contd.)

- **Data can be entered into an array as:**

  var MyArray=new Array()

  MyArray[0]="Paul"

  MyArray[1]="Sam"

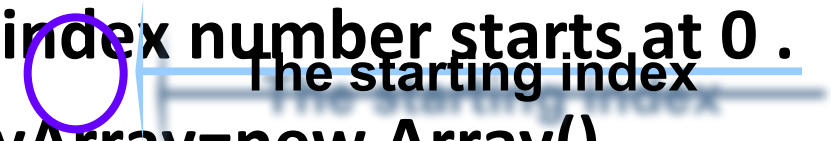
  MyArray[2]="Niel"

- **Data can also be entered into an array as:**

  var MyArray=new Array("Paul","Sam", "Niel")

Arrays (contd.)

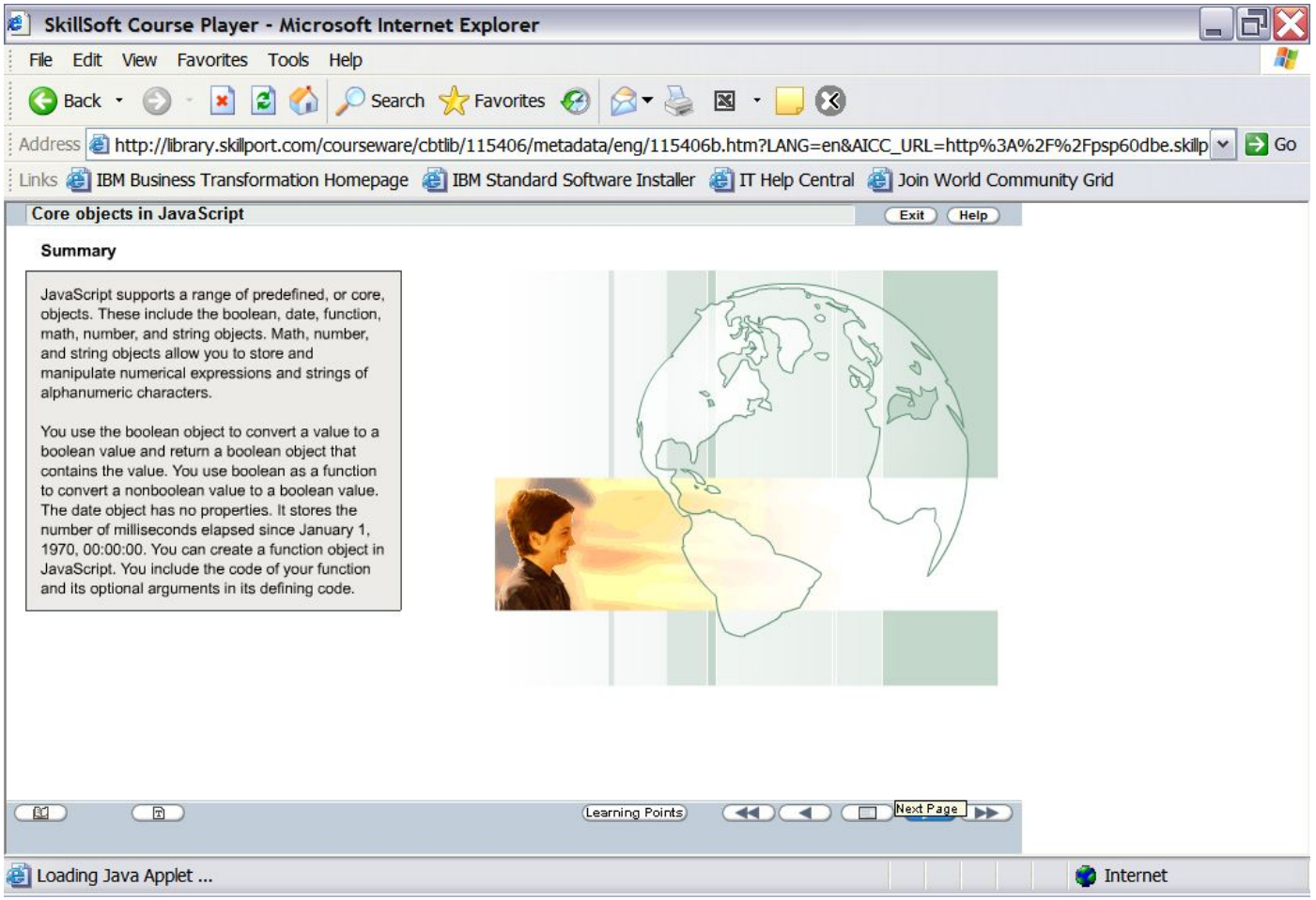
- **Accessing Arrays**
  - **You can refer to a particular element in an array by referring to the name of the array and the index number.**
  - **The index number starts at 0 .**



var MyArray=new Array()

Myarray [0]

Summary


SkillSoft Course Player - Microsoft Internet Explorer
File Edit View Favorites Tools Help
Back Search Favorites
Address http://library.skillport.com/courseware/cbtlib/115406/metadata/eng/115406b.htm?LANG=en&AICC_URL=http%3A%2F%2Fpsp60dbe.skillp
Go
Links IBM Business Transformation Homepage IBM Standard Software Installer IT Help Central Join World Community Grid
Core objects in JavaScript
Exit Help
Summary
JavaScript supports a range of predefined, or core, objects. These include the boolean, date, function, math, number, and string objects. Math, number, and string objects allow you to store and manipulate numerical expressions and strings of alphanumeric characters.
You use the boolean object to convert a value to a boolean value and return a boolean object that contains the value. You use boolean as a function to convert a nonboolean value to a boolean value. The date object has no properties. It stores the number of milliseconds elapsed since January 1, 1970, 00:00:00. You can create a function object in JavaScript. You include the code of your function and its optional arguments in its defining code.
Learning Points
Next Page
Loading Java Applet ...
Internet

Document Object Model

Dipendu Mishra

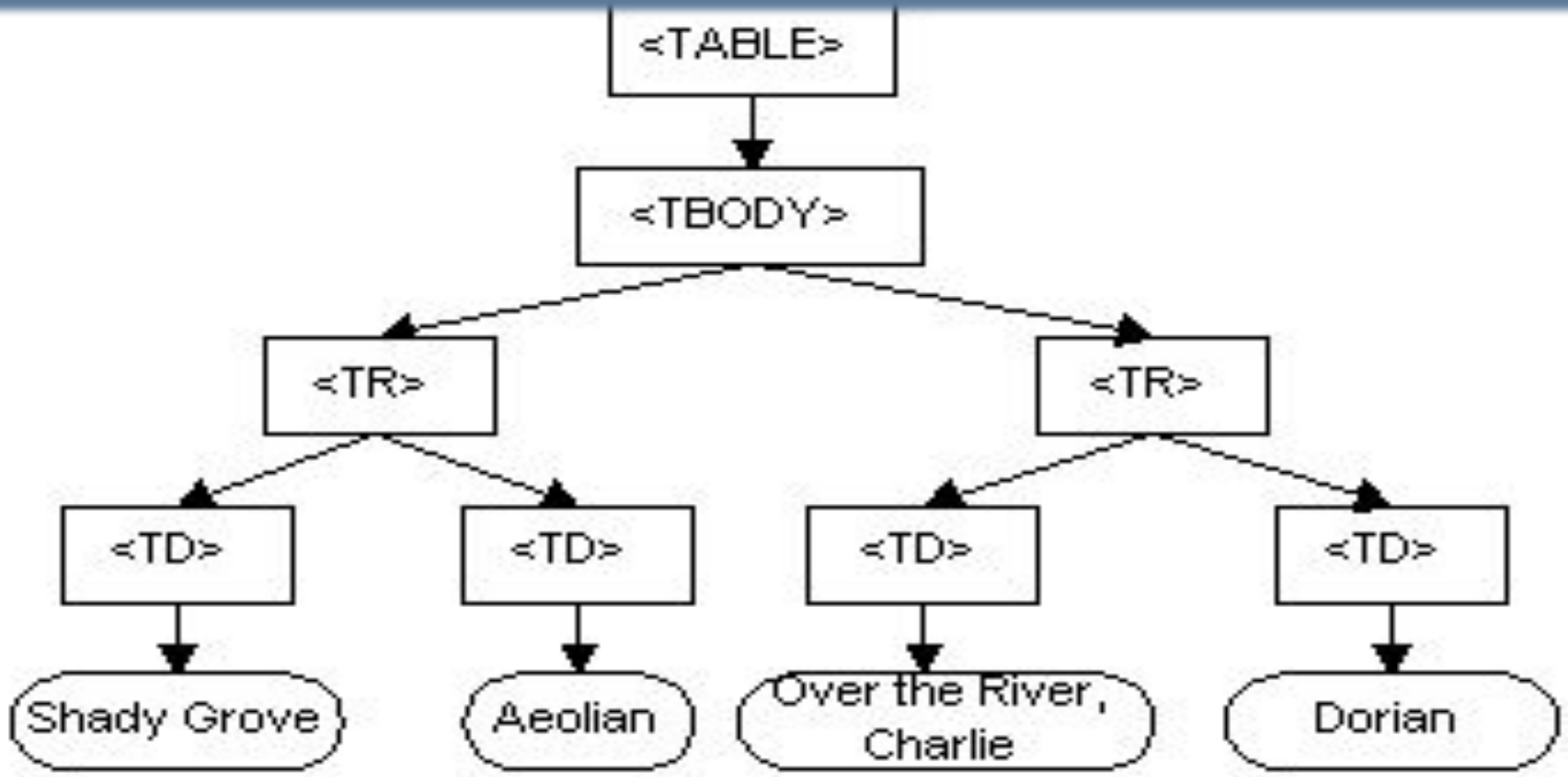
Document Object Model

- **The DOM is a programming *API* for documents.**
- **It is based on an object structure that closely resembles the structure of the documents it *models*.**
- **For instance, consider this table, taken from an HTML document:**

```
<TABLE>
<TBODY>
<TR> <TD>Shady Grove</TD> <TD>Aeolian</TD> </TR>
<TR> <TD>Over the River, Charlie</TD> <TD>Dorian</TD> </TR>
</TBODY>
</TABLE>
```

Document Object Model (contd.)

Graphical representation of the DOM of the example table

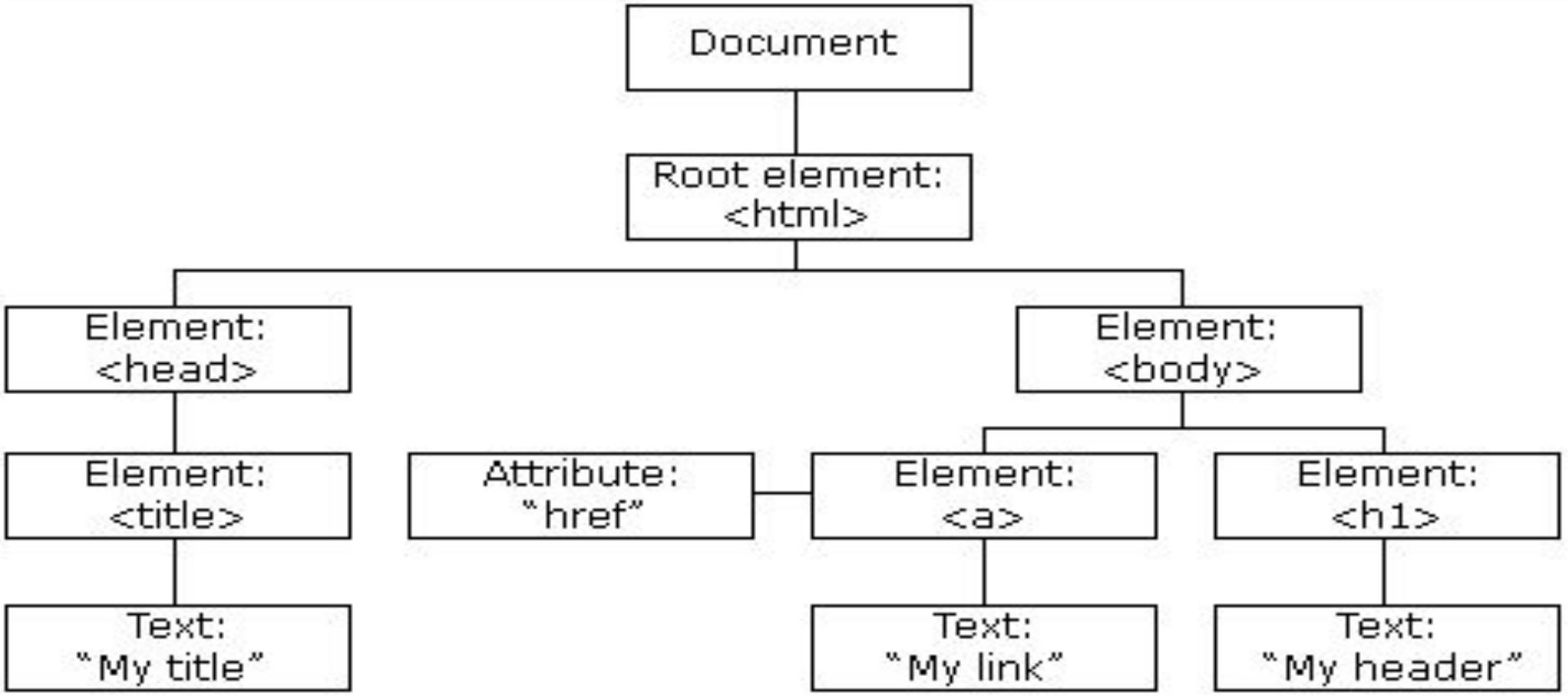


Document Object Model (contd.)

- With JavaScript you can restructure an entire HTML document. You can add, remove, change, or reorder items on a page.
- To change anything on a page, JavaScript needs access to all elements in the HTML document. This access, along with methods and properties to add, move, change, or remove HTML elements, is given through the Document Object Model (DOM).
- In 1998, W3C published the Level 1 DOM specification. This specification allowed access to and manipulation of every single element in an HTML page.
- All browsers have implemented this recommendation, and therefore, incompatibility problems in the DOM have almost disappeared.

Document Object Model (contd.)

Document Tree



How to access the nodes in an HTML

- You can find the element you want to manipulate in several ways:
  - By using the getElementById() and getElementsByTagName() methods
  - By using the parentNode, firstChild, and lastChild properties of an element node

Navigator Object

- **Navigator object is the object representation of the client internet browser or web navigator program that is being used.**
- **This is a top level object to all others object in DOM hierarchy.**

Navigator Object Properties

- **appCodeName – The name of the browser's code such as *Internet Explorer***
- **appName – The name of the browser.**
- **appVersion – The version of the browser.**
- **plugins – An array of plug-ins supported by the browser and installed on the browser.**
- **cpuClass – The type of CPU which may be "x86".**
- **cookieEnabled – A boolean value of true or false depending on whether cookies are enabled in the browser.**

Navigator Object Methods

- **javaEnabled() – Returns a boolean telling if the browser has JavaScript enabled.**

Window Object

- The Window object is the top level object in the JavaScript hierarchy.
- The Window object represents a browser window.
- A Window object is created automatically with every instance of a <body> or <frameset> tag.

Window Object Collections

- **Frames[] – Returns all named frames in the window.**

Window Object Properties

- name – sets of return the name of the window.
- status – sets or returns the name of the window.
- length – sets or returns the number of frames in the window.
- self – returns a reference to the current window.
- statusbar - sets whether or not the browser's statusbar should be visible.
- toolbar - sets whether or not the browser's toolbar should be visible.
- closed – Returns all named frames in the window
- document
- history

Window Object Methods

- **open() – Opens a new browser window.**
- **createPopup() – Creates a popup window.**
- **setInterval(*code,millisec[,lang]*) – Evaluates an expression at specified intervals.**
- **clearInterval(id_of_setInterval) – cancels a timeout that is set with the setInterval() method.**
- **setTimeout(*code,millisec[,lang]*) – Evaluates an expression after a specified number of milliseconds.**
- **clearTimeout(id_of_setTimeout) – cancels a timeout that is set with the setTimeout() method.**
- **focus() – sets the focus to the current window.**
- **blur() – removes focus from the current window.**
- **close() – closes the current window.**

Frame Object

- Frame object represents an HTML frame.
- For each instance of a <frame> tag in an HTML document, a Frame object is created.

Frame Object (contd.)

Frames enable you to display multiple HTML pages within the same browser window.

In

Document Object Model

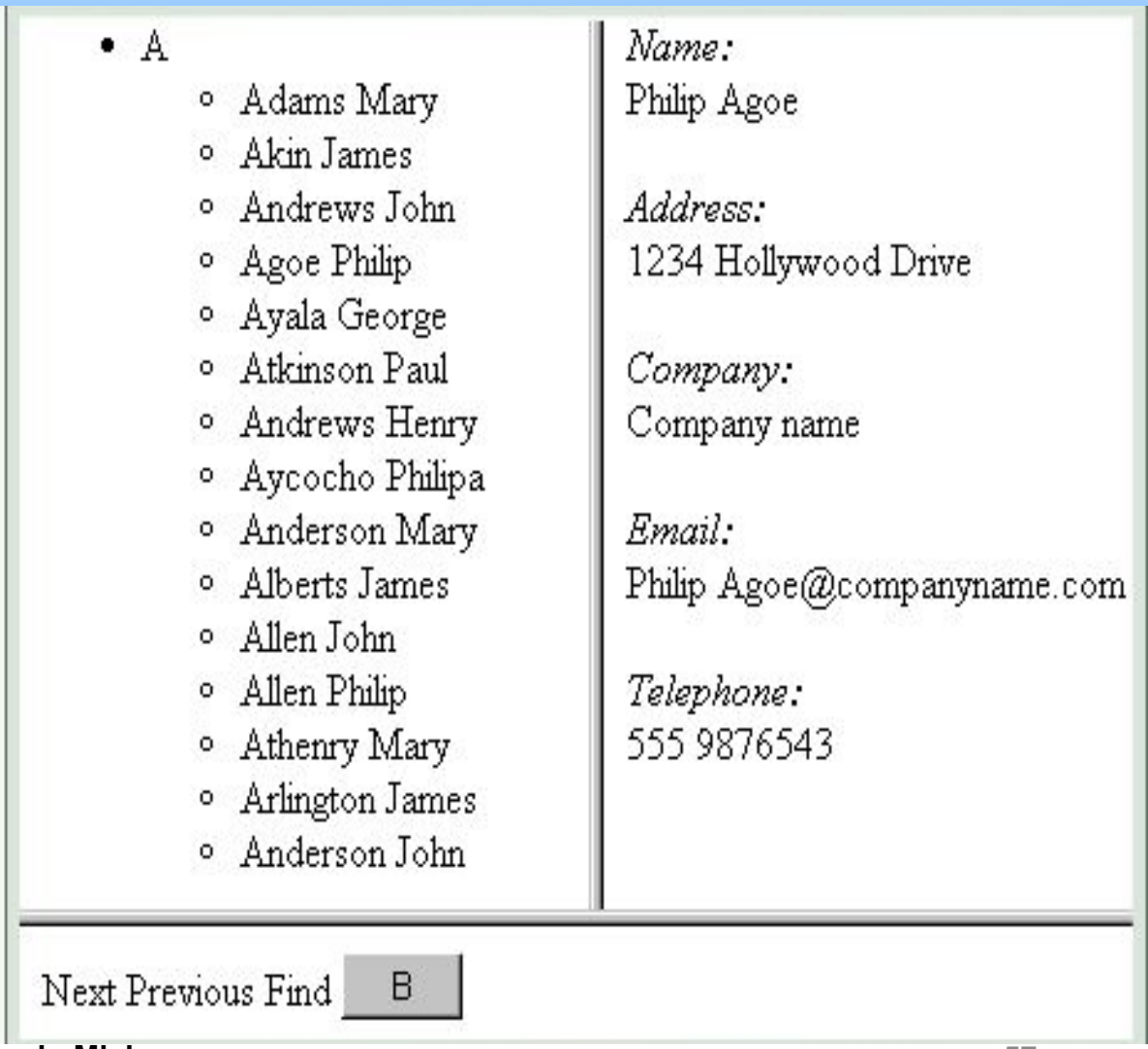
Frames are instances of Window object



Frame Object (contd.)

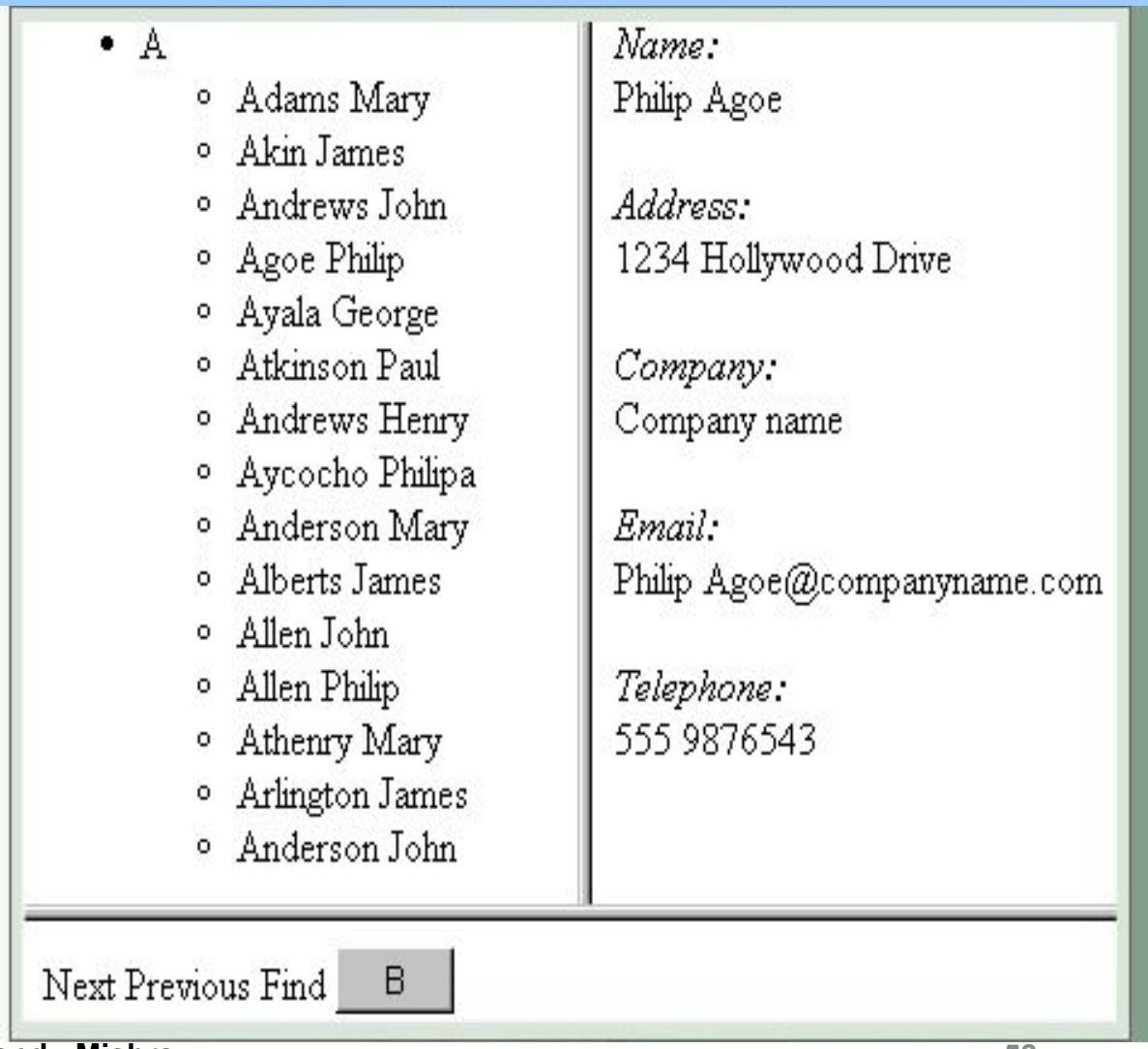
Frames enable you to display multiple HTML pages within the same browser window.

This example shows three such pages, each one being a frame.



Frame Object (contd.)

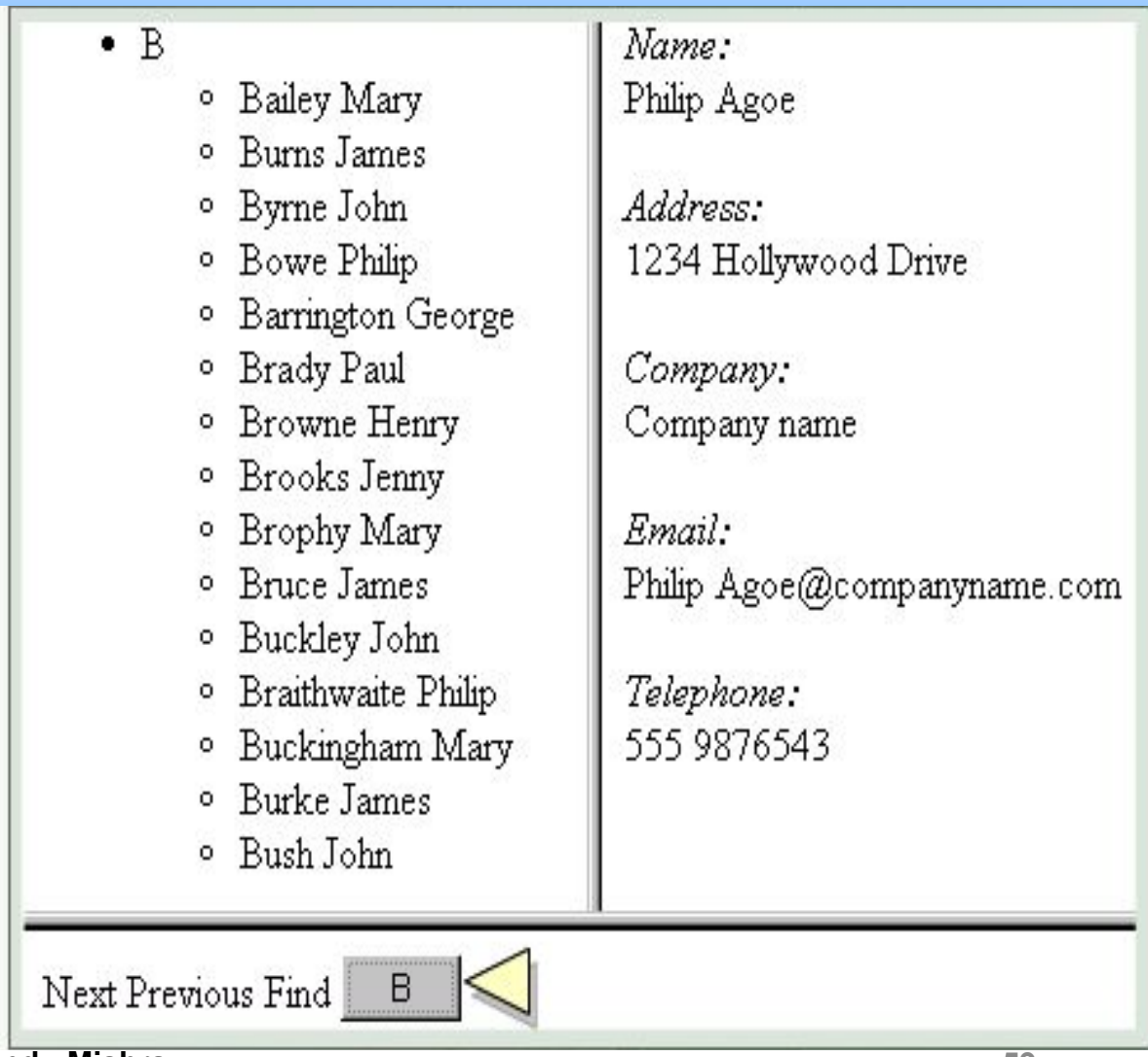
Let's say that you want to provide a button that the user can click to display a set of names. The frames currently display the list of names beginning with the letter A.



Frame Object (contd.)

Let's say that you want to provide a button that the user can click to display a set of names. The frames currently display the list of names beginning with the letter A.

When the user clicks the **B** button, the names frame is updated, and the set of names beginning with the letter B is displayed.

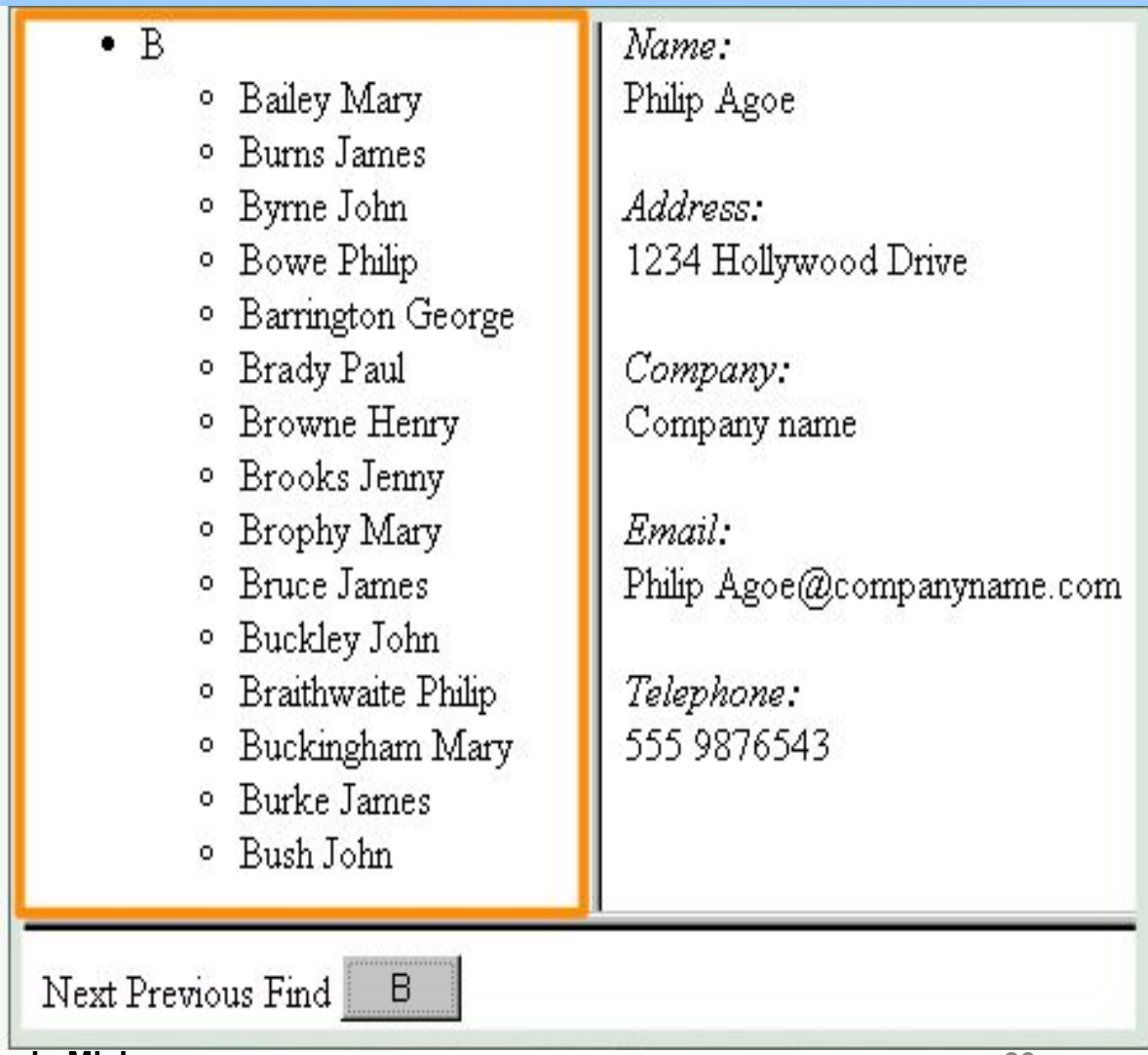


Frame Object (contd.)

Let's say that you want to provide a button that the user can click to display a set of names. The frames currently display the list of names beginning with the letter A.

When the user clicks the **B** button, the names frame is updated, and the set of names beginning with the letter B is displayed.

When a frame is updated, a new frame replaces the old frame in the window.



Frame Object (contd.)

JavaScript automatically indexes the frames as 0, 1, 2, and so on, to match the order in which you enter them in the frameset.

```
// namesFrame is frames[0]
// detailsFrame is frames[1]
// navigateFrame is frames[2]

<HTML>

<FRAMESET ROWS="90%,10%">
        <FRAMESET COLS="30%,70%">
                <FRAME SRC=names.html NAME="namesFrame">
                <FRAME SRC=details.html NAME="detailsFrame">
        </FRAMESET>
        <FRAME SRC=navigate.html NAME="navigateFrame">
</FRAMESET>

</HTML>
```

Frame Object (contd.)

This is the code you use to enable the **B** button to update the names frame with the names_b frame.

```
// navigate.htm
// Update 'A' surnames with 'B' surnames

<INPUT TYPE="button" Value="  B  "
onClick="top.frames[0].location='names_b.html'">
```

Frame Object (contd.)

This is the code you use to enable the **B** button to update the names frame with the names_b frame.

Because you have used the index 0 in the top.frames statement, names_b.html updates names.html.

```
// navigate.htm
// Update 'A' surnames with 'B' surnames

<INPUT TYPE="button" Value="  B  "
onClick="top.frames[0].location='names_b.html'">
```

```
// namesFrame is frames[0]
// detailsFrame is frames[1]
// navigateFrame is frames[2]

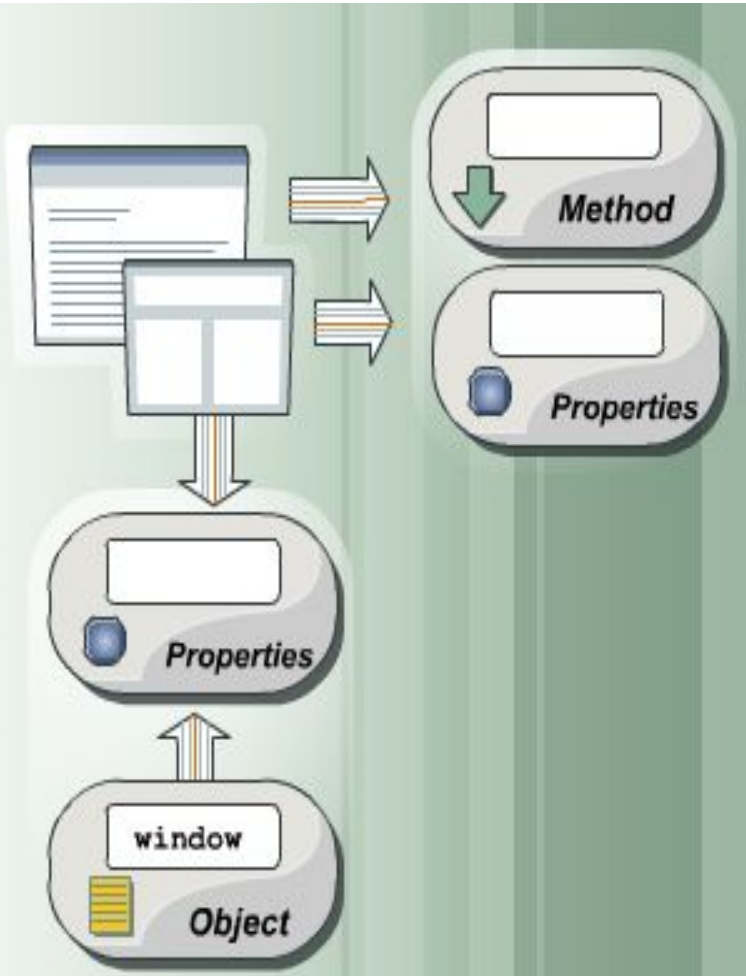
<HTML>

<FRAMESET ROWS="90%,10%">
        <FRAMESET COLS="30%,70%">
                <FRAME SRC=names.html NAME="namesFrame">
                <FRAME SRC=details.html NAME="detailsFrame">
        </FRAMESET>
        <FRAME SRC=navigate.html NAME="navigateFrame">
</FRAMESET>

</HTML>
```

Frame Object (contd.)

Frames are windows – they share the same properties and methods – but a frame is also a property of the window object.

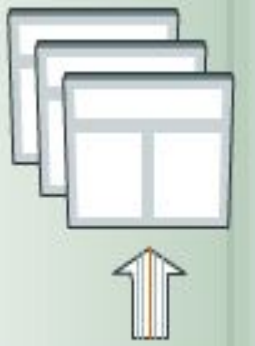


Frame Object (contd.)

Frames are windows – they share the same properties and methods – but a frame is also a property of the window object.

Because a frame is a window, a frame can also be a property of another frame object.

So, for example, you use this code to refer to the first subframe of the third frame object.



```
frame[2].frame[0]
```

Frame Object (contd.)

You can use either of these lines of code to refer to the same frame.

```
//Each of these lines of code uses parent in place of the window
//name

parent.framename
parent.frame[index]
```

Frame Object (contd.)

You can use either of these lines of code to refer to the same frame.

For example, you can use this code to enable a sibling to refer to the frame of the parent window named Frame3.

```
//Each of these lines of code uses parent in place of the window
//name

parent.framename
parent.frame[index]
```

```
parent.Frame3
```

Frame Object (contd.)

You can use either of these lines of code to refer to the same frame.

For example, you can use this code to enable a sibling to refer to the frame of the parent window named Frame3.

Or, if it is the third frame of the current window, you can refer to it as frames[2].

```
//Each of these lines of code uses parent in place of the window
//name

parent.framename
parent.frame[index]
```

```
parent.Frame3
```

```
parent.frames[2]
```

Test Your Understanding

Question

Which of the following happens when a frame is updated?

1. A new frame replaces the old frame in the window
2. All the window's frames are updated
3. The data in the frame is updated

Test Your Understanding

Question

Which of the following happens when a frame is updated?

1. A new frame replaces the old frame in the window ✓
2. All the window's frames are updated
3. The data in the frame is updated

That's right.

When a frame is updated a new frame replaces the old frame in the window.

Document Object

- The document object represents the entire HTML document and can be used to access all elements in a page.
- The document object is part of the window object and is accessed through the window.document property or simply document.

Document Object Collections

- anchors[] - Returns a reference to all Anchor objects in the document.
- forms[] - Returns a reference to all Form objects in the document.
- images[] - Returns a reference to all Image objects in the document.
- links[] - Returns a reference to all Area and Link objects in the document.

Document Object Properties

- **Body – gives direct access to the <body> element.**
- **Cookie – Sets or returns all cookies associated with the current document.**
- **lastModified – Returns the date and time a document was last modified.**
- **Title – Returns the title of the current document.**
- **URL – Returns the URL of the current document.**

Document Object Methods

- write() - Writes HTML expressions or JavaScript code to a document
- writeln() – Similar to write(), with the addition of writing a new line character after each expression.
- open() - Opens a stream to collect the output from any document.write() or document.writeln() methods.
- close() - Closes an output stream opened with the document.open() method, and displays the collected data
- getElementByID() – Returns a reference to the first object with the specified id.
- getElementsByName() – Returns a collection of objects with the specified name.
- getElementsByTagName() – Return a collection of objects with the specified tag name.

Location Object

- Location object is an JavaScript object it is not an DOM object.
- The Location object is automatically created by the JavaScript runtime engine and contains information about the current URL.
- The Location object is part of the Window object and is accessed through the window.location property.

Location Object Properties

- **href – It contains the complete URL of the currently displayed document.**
- **protocol – specifies the protocol to be used with the document.**

Example

window.location.href = http://www.ibm.com

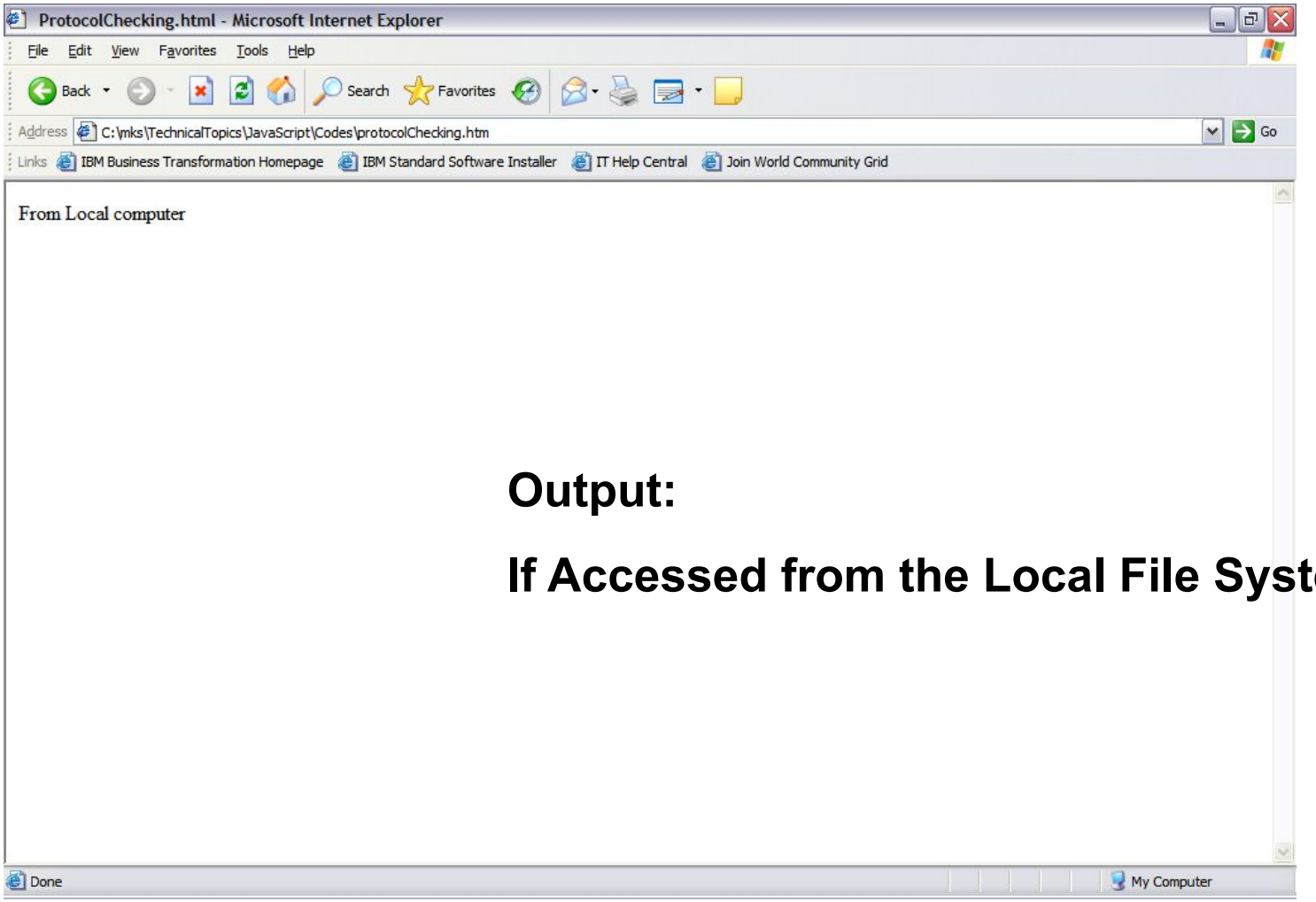
It sets the URL address of the current

Location Object Methods

- **Assign (url) – It loads a new document.**
- **Reload() – the current document.**
- **Replace() – Replaces the current document with a new one.**

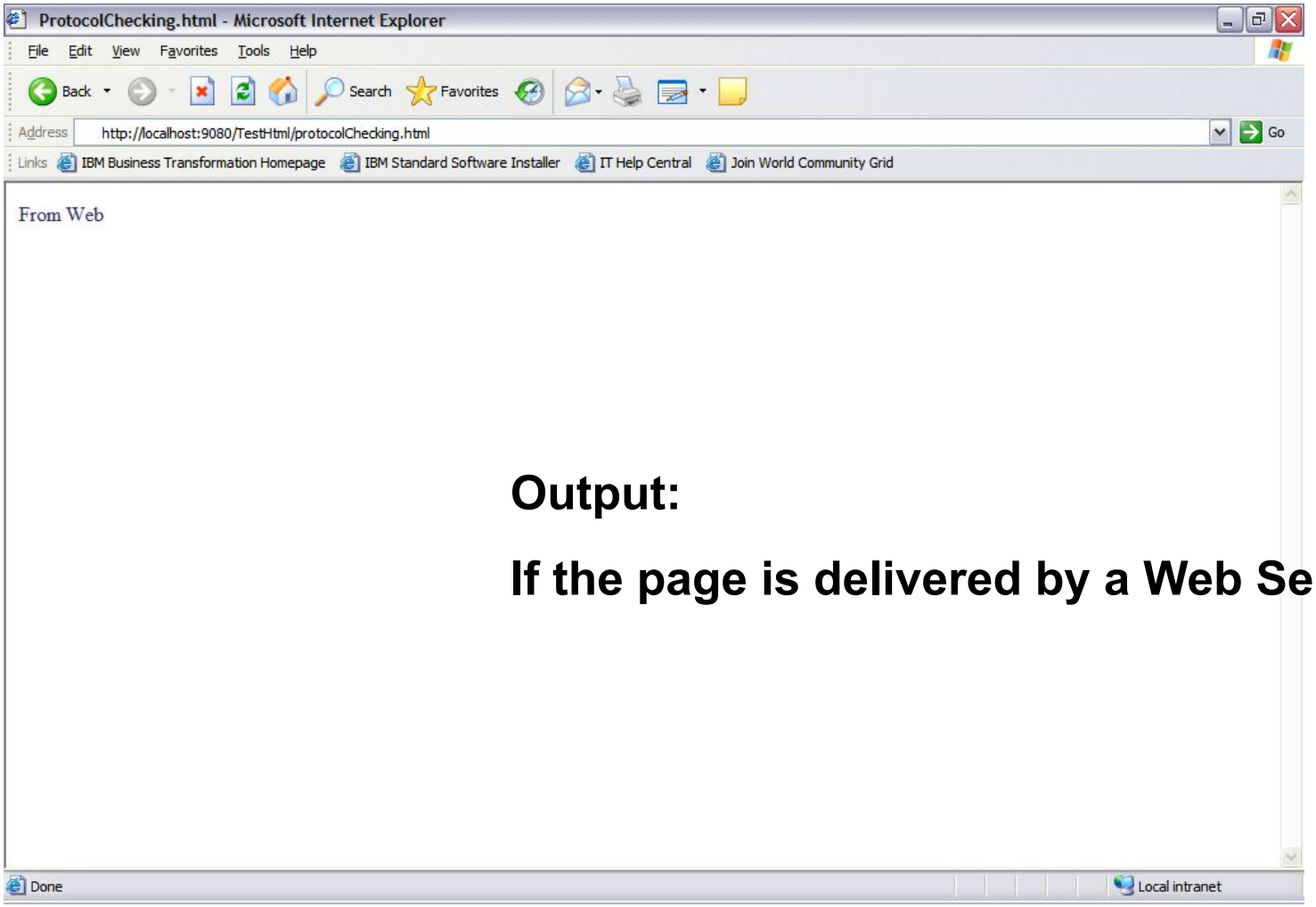
Location Object Example

```
<html>
<body>
<script>
switch (window.location.protocol)
{
  case "http:":
    document.write("From Web<BR>\n")
    break
  case "file:":
    document.write("From Local computer<BR>\n")
    break
  default:
    document.write("Unknown Source<BR>\n")
    break
}
</script>
</body>
</html>
```



Output:

If Accessed from the Local File System



Output:

If the page is delivered by a Web Server.

History Object

- Windows and Frames maintain an array that enables the user to navigate back and forth between URLs previously viewed in the current browser session. This array is called the History object.
- History object is a JavaScript object.
- The History object is automatically created by the JavaScript runtime engine and consists of and array of URLs.
- History object is a window property.
- It is a part of window object & accessed through window.history property.

History Object (contd.)

- The two most useful methods of history object are:
  - ✓ back() – enables the user to see the previous URL in the history array.
  - ✓ forward() - enables the user to see the next URL in the history array.

Test Your Understanding

Question

What is the function of the history object?

1. It enables the user to open any previously visited site in the browser
2. It enables the user to navigate back and forth between URLs previously viewed in the current browser session
3. It enables you to give keyboard focus to the top-level window

Test Your Understanding

Question

What is the function of the history object?

That's right.

The history object enables the user to navigate back and forth between URLs previously viewed in the current browser session.

1. It enables the user to open any previously visited site in the browser
2. It enables the user to navigate back and forth between URLs previously viewed in the current browser session ✓
3. It enables you to give keyboard focus to the top-level window

Image Object

- Image object represents an embedded image.
- For each instance of an <img> tag in an HTML document, an Image object is created.

Image Object Properties

- **Name – Sets or returns the name of an image.**
- **align – Sets or returns how to align an image according to the surrounding text.**
- **Alt – Sets or returns an alternate text to be displayed, if a browser cannot show an image.**
- **Border – Sets or returns the border around an image.**
- **Height & width – Sets or returns the height & width of an image.**
- **Src – Sets or returns the URL of an image.**
- **useMap - sets or returns the value of the usemap attribute of an image.**

Link Object

- The Link object represents an HTML <link> element.
- The <link> element defines the relationship between two linked documents.
- The <link> element is defined in the head section of an HTML document.

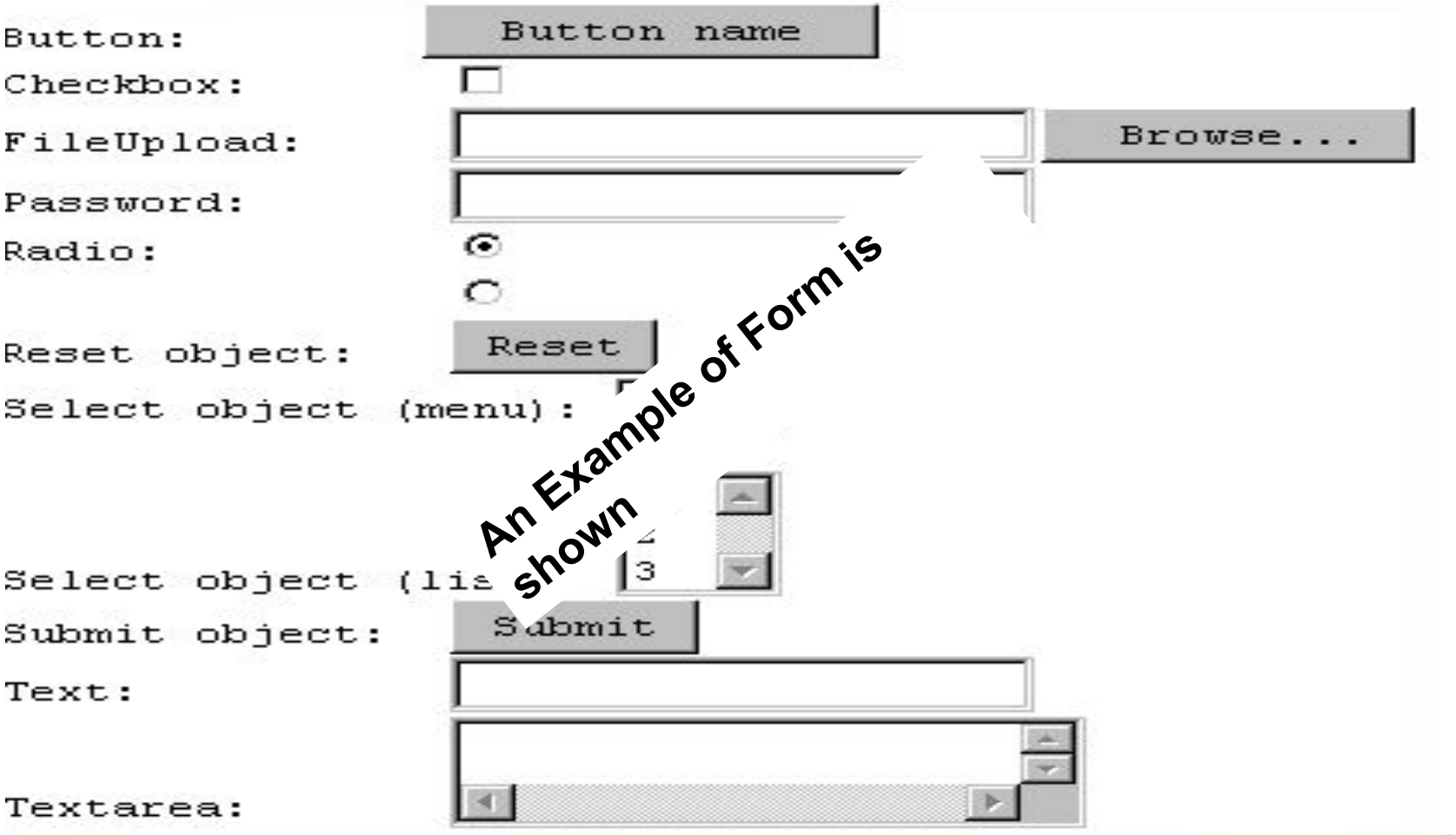
Link Object Properties

- **Href – Sets or returns the URL of the linked resources.**
- **Id – Sets or returns the id of a <link> element.**
- **Name – Sets or returns the name of a <link> element.**
- **Type – Sets or returns the MIME type of the target URL (like "text/css",text/javascript","image/gif",etc.).**

Form Validation
&
Form Object Model

Dipendu Mishra

Form Object Model



Form Object Model (contd.)

- In Java Script, an HTML form is represented by the form object.
- Forms exist as elements of the Forms array. This array is a property of the document object.
- The elements of the form like textbox, button etc. are stored as an forms object elements array property.

Form Object Model (contd.)

This statement refers to the first form in your document.	→	document.forms[0]
This statement refers to the third element in the first form array	→	document.forms[0].elements[2]
You can refer the form by its names also.	→	document.f1 – Assuming f1 as the form name

Test Your Understanding

Question

Identify the correct statement about the Elements array.

1. The form's NAME attribute is stored in the element array
2. The Elements array is a property of the form object
3. Elements are stored in alphabetical order

Test Your Understanding

Question

Identify the correct statement about the Elements array.

That's right.

The elements array is a property of the form object.

1. The form's NAME attribute is stored in the element array
2. The Elements array is a property of the form object ✓
3. Elements are stored in alphabetical order

Form Object Model (contd.)

- The Form object has two methods:
  - ✓ submit()
  - ✓ reset()

Example

```
document.forms[0].submit()
OR
document.forms[0].reset()
```

Form Object Model (contd.)

- Some of the properties of Form are as follows:
  - ✓ action - specifies the server side script URL where the form will be submitted.
  - ✓ method – specifies the method to be used to submit a form.
  - ✓ target - It is used to load the result of the form submission in a target window.

Example

```
document.forms[0].action = "URL"

document.forms[0].method = "Get"

document.forms[0].target =
    "window.open('','FormWin','resizable=no,width=200,height=200')"
```

Summary

When you are writing a JavaScript program, you will often want to invite the user to input data. To enable your form to accept input from the user, you must include elements such as text boxes or buttons on it. The Elements array is a property of the forms object. As you place elements on your form, they become entities in the Elements array. You can refer to an element in the Elements array by name.

The JavaScript form object has a range of properties that enable you to enhance user interactivity. Its action, method, and target properties correspond directly to the HTML ACTION, METHOD, and TARGET attributes of the <FORM> tag.



Cookies

Dipendu Mishra

Cookies

- A cookie can store a small amount of information about a user visiting a site.
- A cookie is a small text file that is stored on the site visitor's computer by their browser .
- Because the cookie is stored on the user’s computer, it does not require any server space no matter how many users you have .
- With JavaScript, you can both create and retrieve cookie values.

Cookies (contd.)

- You can use cookies to :
  - save user preferences.
  - customize data.
  - remember the last visit.
  - Keep track of items in an order while a user browses.
  - Remember the information your site's visitors gave last time.
- Cookies can be created, read and erased by JavaScript. They are accessible through the property

Cookies (contd.)

- What does a cookie contain?
  - Name value pair
    - The first element in a cookie is a "name" attribute, followed by the data to be stored in the cookie.
  - The expiry date
    - specifies how long the cookie is valid for.

Cookies (contd.)

- What does a cookie contain?
  - The path
    - this states where the cookie may be accessed from on the Web site. Most often, this is set to the server's document root.
  - The domain
    - The "domain" attribute allows you to set a domain name for the cookie. Again, this is optional.

Question

The name-value pair is the _____ element in the Cookie

- first
- second
- third

Answer

The name-value pair is the _____ element in the Cookie

- first
- second
- third

Cookies and Security

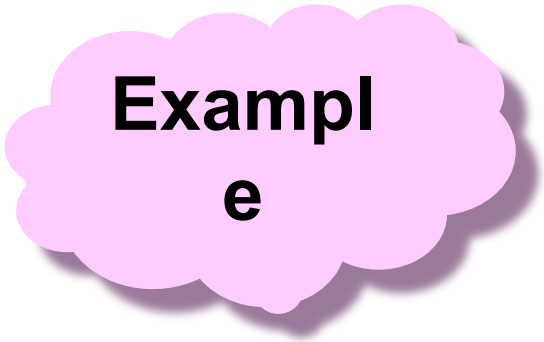
- Turn up security level on your browser to disable cookies or prompt for cookie.
- Delete the content of a cookie and then write protect it.
- Use 3rd party software:
  - Cookie Pal, CookieMaster, CookieCrusher to monitor, browse and edit cookies.

Format of Cookie

- First the name value pair.
- Then a semicolon and a space.
- Then the expiry date.
  - Expiry date should be in UTC format.
- Then again a semicolon and a space.
- Then the path.

Format of Cookie (contd.)

- document.cookie=<name of cookie>=<value of cookie>;<blank space>expires=<date in UTC format>;<blank space>path=<path location>




```
document.cookie = ‘user=Paul; expires=Thu, 2 Aug 2008 20:47:11 UTC; path=/'
```

Example of Setting a Cookie

```
<html>
<head>
<script language="JavaScript">
function setCookie(name, value) {

var exp=new Date("January 31,2008")
document.cookie=name+"="+escape
(value)+"; expires="+exp.toGMTString()
      +"; path=/test“
document.write(“Cookie has been set”)
)
</script>
</head>
<body>
<form>
    <input type="button" value="Set a
    Cookie"
    onClick="setCookie(‘user',‘Paul
    Smith')">
    </form>
</body>
</html>
```

What does the escape function do?

Set an expiry date

escape() function

- There's no escape!
  - Strictly speaking, we should be *escaping* our cookie values -- encoding non-alphanumeric characters such as spaces and semicolons.
  - This is to ensure that our browser can interpret the values properly.
  - Fortunately this is easy to do with JavaScript's escape() function.

  For example

  document.cookie = "username=" + escape(“Paul Smith") + ";
  expires=15/02/2003 00:00:00";

Example of Setting a Cookie

```
<html>
<head>
<script language="JavaScript">
function setCookie(name, value) {

var exp=new Date("January 31,2008")
document.cookie=name+"="+escape
(value)+"; expires="+exp.toGMTString()
      +"; path=/test“
document.write(“Cookie has been set”)
)
```

```
</script>
</head>
<body>
<form>
      <input type="button" value="Set
      a Cookie"
      onClick="setCookie(‘user',‘Paul
      Smith')">
</form>
</body>
</html>
```

The value stored is

user=Paul Smith

Setting the cookie

Reading a Cookie

```
<html>
<head>
<script language="JavaScript">
function readCookie() {

    var ca = document.cookie
    document.write(ca)

}
</script>
</head>
<body>
<form><input type="button"
    value="read"
    onClick="readCookie()">
</form>
</body>
</html>
```

Returns the cookie name value pair

Value retrieved is

user=Paul Smith

Extracting only the value from the Cookie

```
<html>
<head>
<script language="JavaScript">
function readCookie(c_name)
{
if (document.cookie.length>0)
 {c_start=document.cookie.indexOf(c_name + "=")
        if (c_start!=-1)
   {
 c_start=c_start + c_name.length+1
     c_end=document.cookie.indexOf(";",c_start)
 if (c_end==-1) c_end=document.cookie.length
 document.write(
      document.cookie.substring(c_start,c_end
      ))
 }
  }
}
 </script>
</head>
<body>
<form> <input type="button" value="read"
     onClick="readCookie('user')">
</form>
</body>
</html>
```

Displays the value:

Paul Smith

Delete Cookies

```
<html>
<head>
<script language="JavaScript">
function deleteCookie ( cookie_name)
{
 var cookie_date = new Date ( )
 cookie_date.setTime (
     cookie_date.getTime() - 1 )
 document.cookie = cookie_name += "=;
     expires=" + cookie_date.toGMTString()
}
 </script>
</head>
```

```
<body>
<form>
<input type="button"
    value="“delete"
    onClick="deleteCookie(‘user')">
</form>
</body>
</html>
```

The cookie's expiry date is set

to one second less then the

current date.

Modifying a cookie

- To modify a cookie simply reset the values and use the same procedure for setting the cookie.
- Ensure that the cookie name is existing or a new cookie will be created.

Summary

- Cookies are small text files that sit on your hard disk.
- Cookies are created when you visit websites that use cookies to store information that they need (or prefer).
- Websites often use cookies to personalize the user experience - such as remembering your name (assuming you supplied it previously) or remembering the items in your shopping cart from previous visits.
- Despite the many misconceptions about cookies being malicious, they are quite harmless.
- Cookies can't give your personal details to other websites, or transmit a virus or anything like that.
- A cookie can only be read by the server that created it.
- Websites normally use cookies to make its users' lives easier, not harder.

Thank You For Your Participation